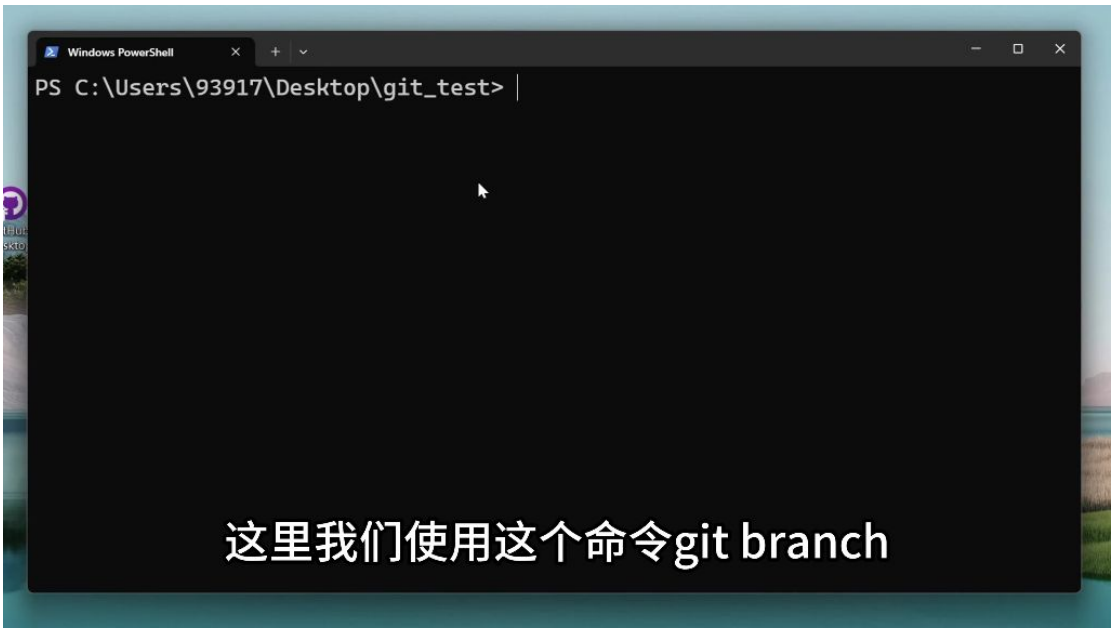
-- 以下为AI生成的图文笔记内容 --

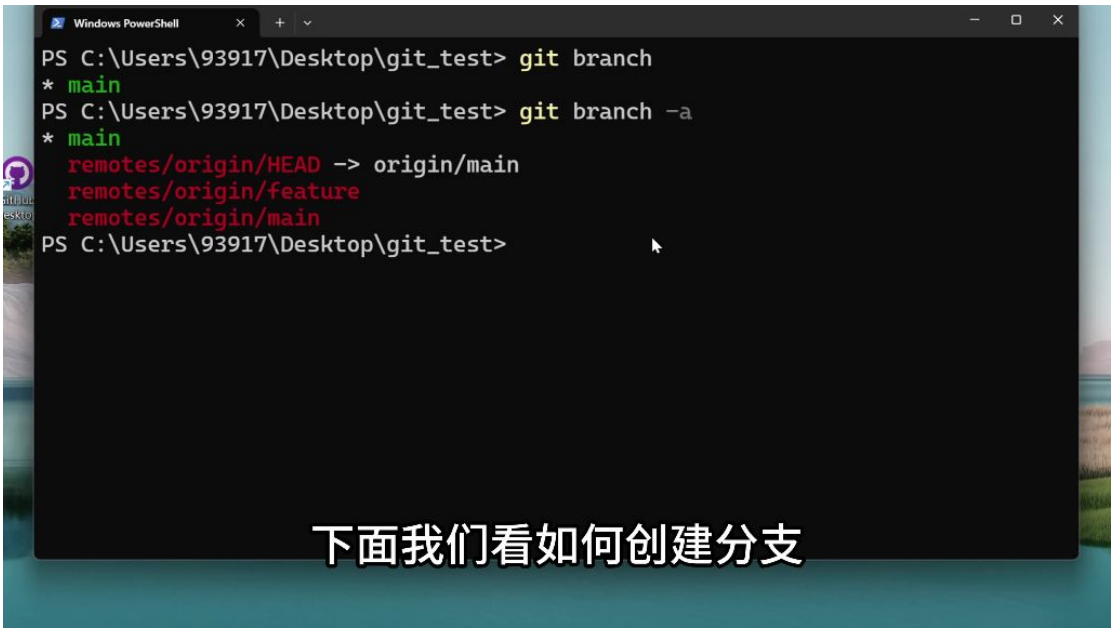
一、*git分支的操作00:01*

1. 查看分支 00:06

- 

- **本地分支查看**: 使用命令git branch可以查看所有本地分支
- **远程分支查看**: 使用git branch -a可以查看所有分支（包括远程分支），其中带remotes/前缀的是远程分支

2. 创建分支 00:25

- 

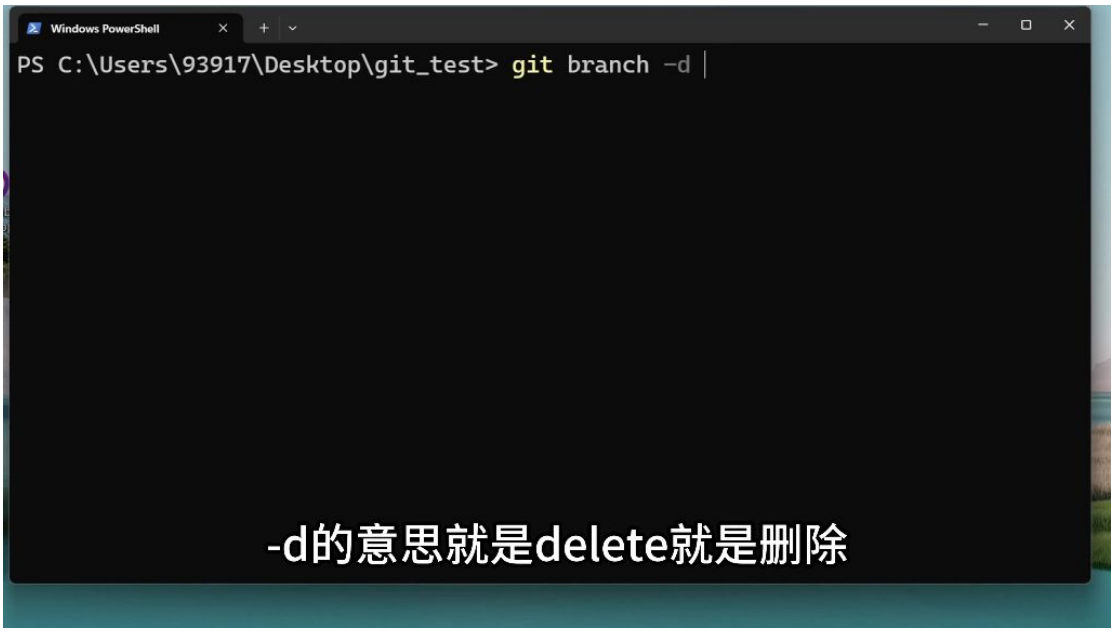
- **创建命令**: git checkout -b <分支名>，其中-b参数表示创建新分支
- **创建基础**: 新分支是基于当前所在分支创建的

- **示例**: 从main分支创建feature2分支：git checkout -b feature2

3. 推送分支 00:52

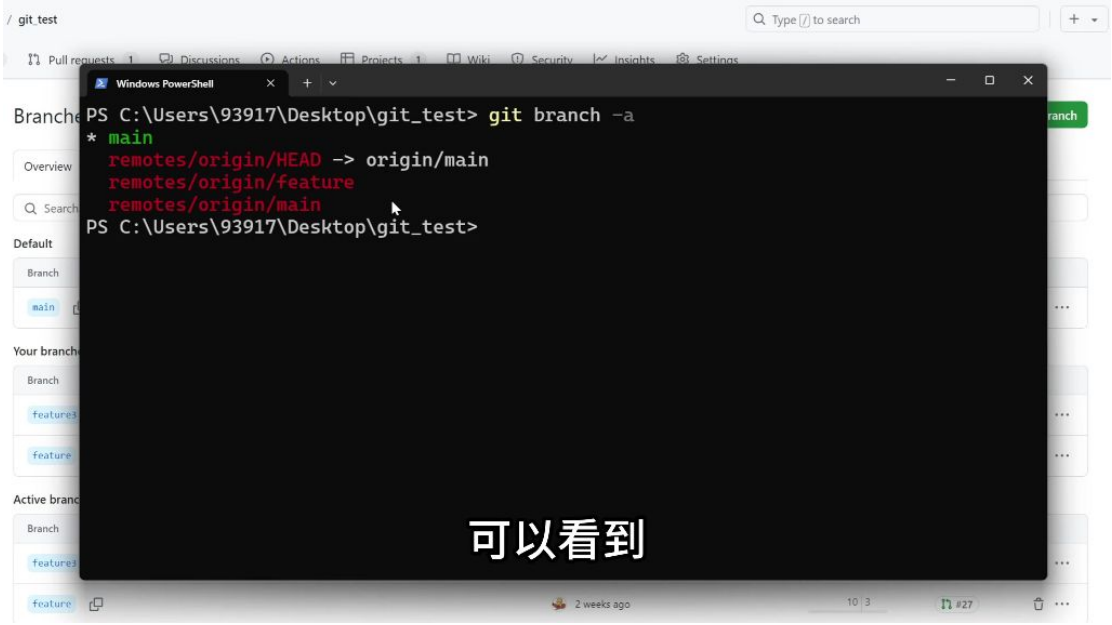
- **推送命令**: git push --set-upstream origin <分支名>
- **作用**: 将本地分支推送到远程并建立关联关系
- **参数说明**: --set-upstream用于建立本地与远程分支的跟踪关系

4. 删除分支 01:12

- 

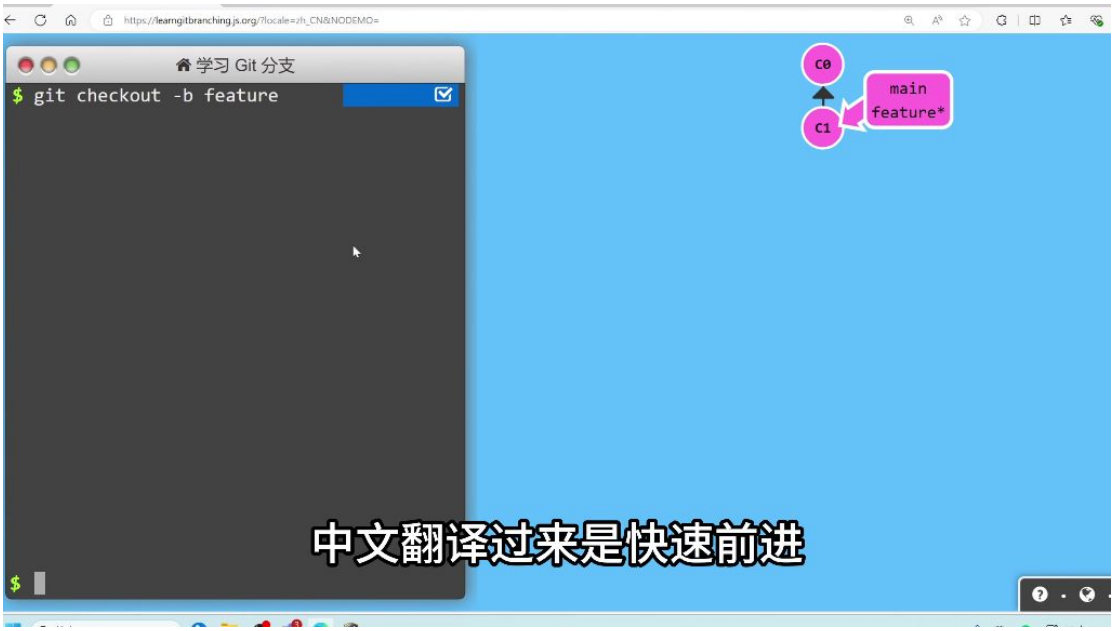
- **本地删除**:
  - 普通删除：git branch -d <分支名>
  - 强制删除：git branch -D <分支名>（用于未合并的分支）
- **远程删除**: git push origin --delete <分支名>
- **注意事项**: 不能删除当前所在分支，需先切换到其他分支

5. 拉取远端分支 02:15

- 

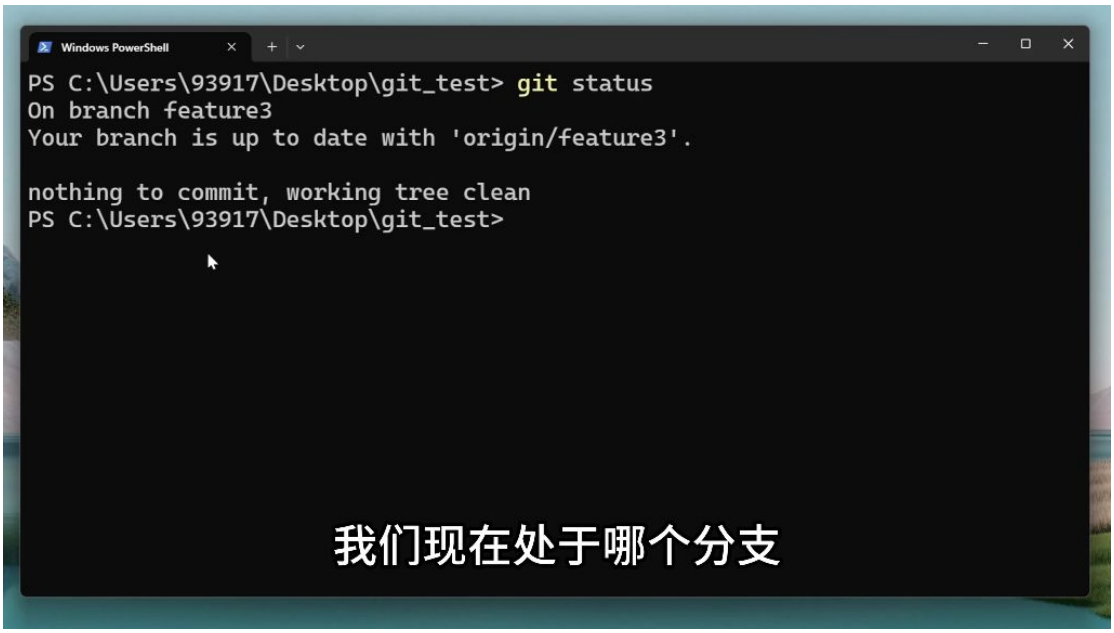
- **同步命令**: 需要先执行git fetch同步远程分支信息
- **检出命令**: 使用git checkout <分支名>将远程分支检出到本地
- **自动关联**: 检出时本地分支会自动与远程同名分支建立关联

6. 快速前进 03:35

- 

- **定义**: 当两个分支没有分叉（一个分支创建后只有该分支有提交）时的合并状态
- **特点**: 合并时只需移动指针，不会产生新的提交记录
- **非快速前进**: 当分支分叉后合并会产生新的提交记录
- **强制非快速前进**: 使用git merge --no-ff即使可以快速前进也会产生新提交

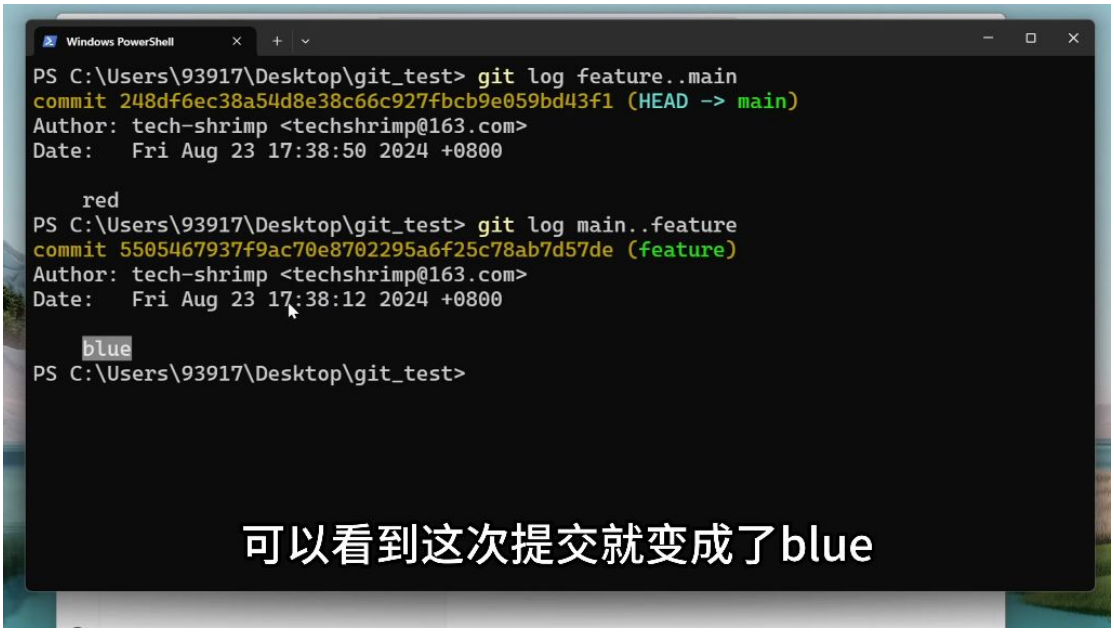
7. rebase 06:22

- 

- **基本用法**: git rebase <目标分支>将当前分支变基到目标分支
- **与merge区别**: 会重写提交历史，使提交线更整洁
- **推送注意**: 通常需要强制推送，但在fast-forward状态下可普通推送

8. 比较分支不同 07:36

1）git log 07:46

- 

- **两点比较**:
  - git log A..B: 显示B有而A没有的提交
  - 顺序重要，比较结果会不同

- **三点比较**: git log A...B显示两分支不同的所有提交

2）git diff 08:38

- **文件比较**:
  - git diff A..B: 比较两分支文件差异
  - 同样顺序会影响比较结果

9. squash merge 09:16

-

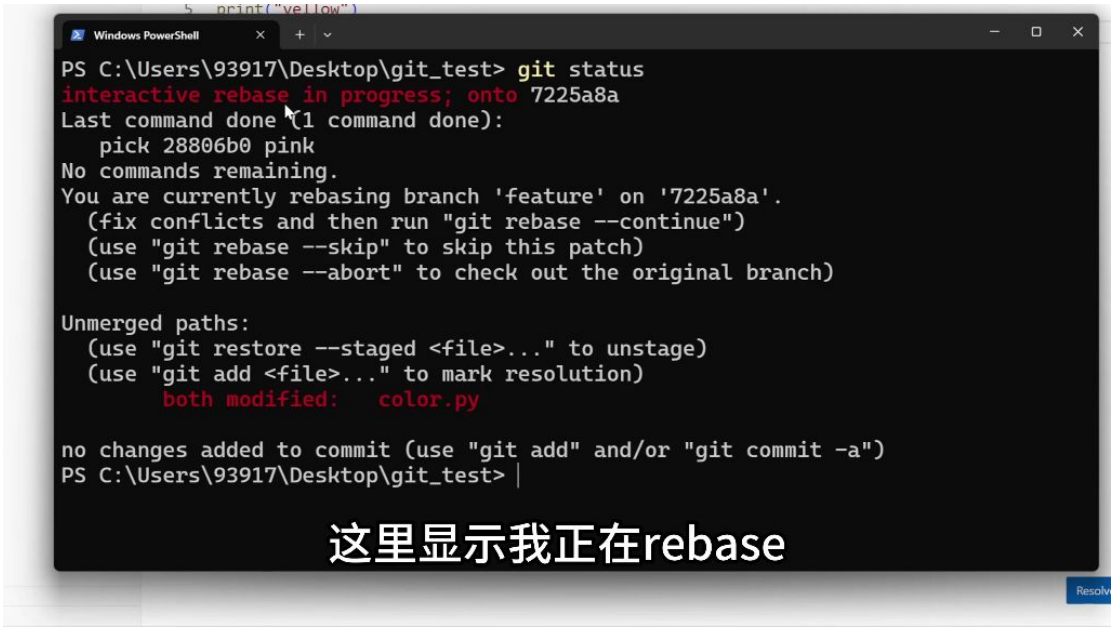
- **作用**: 将多个提交合并为一个后合并
- **命令**: git merge --squash <分支名>
- **特点**: 不会自动提交，需要手动完成最终提交
- **适用场景**: 需要保持主分支提交历史整洁时

10. 解决冲突 10:06

-

- **冲突标识**: 文件中会用<<<<<<<、=======、>>>>>>>标记冲突内容
- **解决步骤**:
  - 手动修改文件保留需要的内容
  - git add标记冲突已解决
  - git commit完成合并
- **提交信息**: 通常使用"resolve conflict"作为提交信息

11. rebase解决冲突 11:32

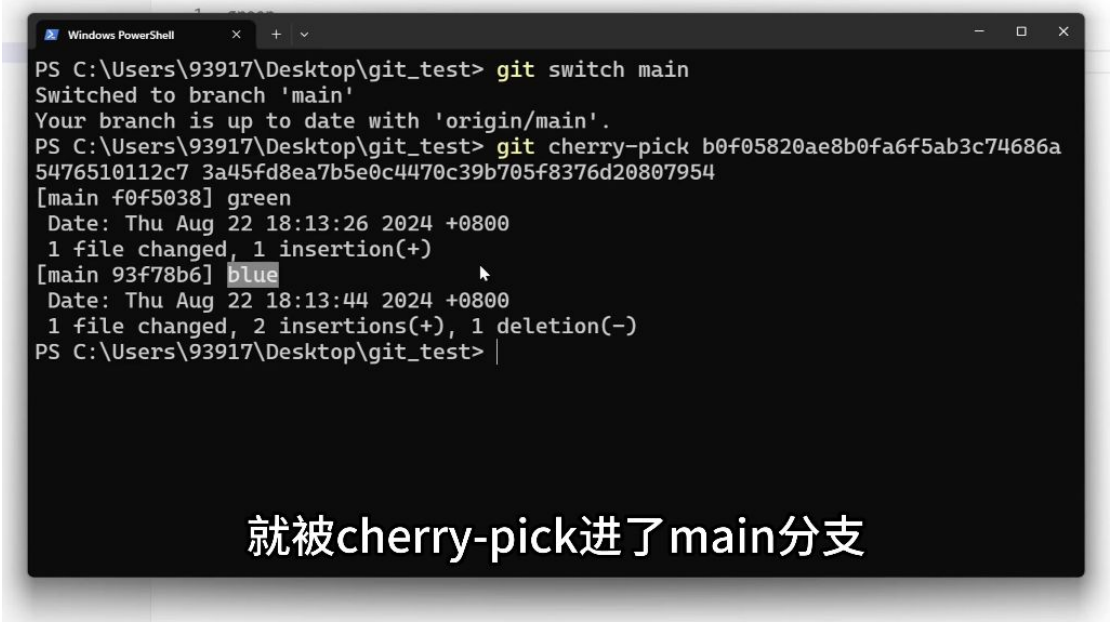
- 

- **冲突处理**:

- 解决冲突文件
- git add标记解决
- git rebase --continue继续变基

- **放弃选项**: 可使用git rebase --abort放弃整个变基操作

12. cherry-pick 12:57

- 

- **作用**: 选择性地将某些提交应用到当前分支
- **命令**: git cherry-pick <commit-id>
- **特点**: 可以只合并部分提交，而不是整个分支
- **使用场景**: 当只需要某个分支的特定修改时

二、知识小结

知识点	核心内容	易混淆点/注意事项	难度系数
查看分支	git branch查看本地分支 git branch -a查看所有分支(含远程)	绿色显示本地分支 带remote/前缀的是远程分支	★
创建分支	git checkout -b <分支名> 基于当前分支创建新分支	创建时会自动切换到新分支	★

推送分支	git push --set-upstream origin <分支名> 将本地分支与远程同名分支关联	首次推送必须使用--set-upstream	★★
删除分支	本地：git branch -d <分支名> 远程：git push origin --delete <分支名>	**未合并分支需用-D强制删除** 不能删除当前所在分支	★★
同步远程分支	git fetch手动同步远程分支信息 git checkout <远程分支名>检出到本地	远程分支更新不会自动同步	★★
普通合并(merge)	git merge <分支名> 将指定分支合并到当前分支	会产生新的合并提交记录	★★★
Fast Forward合并	当分支未分叉时，直接移动指针完成合并	**不会产生新的提交记录** 图示说明分叉与未分叉状态	★★★
非Fast Forward合并	git merge --no-ff <分支名> 强制生成合并提交记录	即使满足FF条件也创建新节点	★★★★
变基(rebase)	git rebase <目标分支> 将当前分支修改"移植"到目标分支最新提交	**多数情况需强制推送** FF状态下可普通推送	★★★★
分支对比	提交对比：git log 分支A..分支B 文件对比：git diff 分支A..分支B	两点与三点表示法区别： ..单边对比 ...双边对比	★★★
压缩合并(squash)	git merge --squash <分支名> 将多个提交压缩为单个提交合并	需手动执行最终commit	★★★★
冲突解决	merge/rebase均可能冲突 1. 编辑冲突文件	rebase冲突后需用git rebase --continue	★★★★

	2. git add标记解决 3. 继续操作(continue)	merge冲突后直接commit	
精选提交(cherry-pick)	git cherry-pick <commitID> 选择性地合并特定提交	可连续pick多个提交 需先切换到目标分支	★★★★